Tarea 5 · TSP con Templado Simulado

15 ciudades · Intercambio de dos posiciones · Minimización

Material: presentación 8, láminas 27–32. Fecha: 3 de noviembre de 2026.

Problema y datos utilizados

Se busca un ciclo que visite las 15 ciudades exactamente una vez y regrese a la ciudad inicial, minimizando su longitud. La solución se representa mediante una permutación vc de las ciudades; la ciudad de inicio se añade únicamente al cerrar el ciclo para evaluarlo o dibujarlo. [1]

Se utilizaron las 15 ciudades sintéticas autorizadas para esta práctica, identificadas como C01–C15. Sus coordenadas enteras se generaron con semilla 42 y se conservan en datos/ciudades.csv. Las longitudes se expresan en unidades del plano, no en kilómetros.

$$d(i,j) = \sqrt{[(x_i - x_j)^2 + (y_i - y_j)^2]}$$

$$L(v) = d(v_0,v_1) + \ldots + d(v_{13},v_{14}) + d(v_{14},v_0)$$

La última suma incluye explícitamente el regreso al origen. La matriz es simétrica; se calcula con distancias euclidianas de doble precisión, sin redondearlas durante la optimización.

Ciudad	x	y	Ciudad	x	y	Ciudad	x	y
C01	9	78	C06	53	98	C11	50	37
C02	66	44	C07	74	76	C12	18	93
C03	43	86	C08	72	79	C13	78	65
C04	8	70	C09	51	12	C14	40	83
C05	20	9	C10	84	45	C15	55	44

Parámetros reproducibles

Parámetro	Valor usado
Temperatura inicial T_0	100.0
Umbral de parada Tmín	0.01
Enfriamiento por nivel	$T \leftarrow 0.99 \cdot T$ (disminución del 1 %)
Propuestas por temperatura	100
Ruta inicial de la corrida principal	Permutación aleatoria; semilla 0
Repeticiones de comprobación	20 corridas, semillas 0 a 19

La ruta inicial, los intercambios y las decisiones probabilísticas usan un generador con semilla. Repetir los mismos datos y parámetros permite reproducir el experimento.

Vecindad y regla de aceptación

En cada propuesta se eligen uniformemente dos posiciones distintas i y j de vc y se intercambian para obtener vn. La vecina conserva las 15 ciudades sin repeticiones. Hay 15 × 14 / 2 = 105 intercambios distintos posibles para una permutación; se propone sólo uno por iteración, como pide el enunciado.

```
i, j = rng.sample(range(15), 2)
vn = vc.copy()
vn[i], vn[j] = vn[j], vn[i]
```

Para minimizar se invierte la resta del exponente de la lámina 27: se define Δ = L(vn) − L(vc). La regla completa sigue la lámina 30: una mejora estricta se acepta siempre; el resto se somete a la probabilidad logística. Esta variante también está documentada por MathWorks. [1–2]

$$p = 1 / (1 + \exp((L(vn) - L(vc)) / T))$$

Situación	Decisión
Δ < 0: la vecina es mejor	Aceptar con probabilidad 1
Δ = 0: igual longitud	Aceptar con probabilidad 0.5
Δ > 0: la vecina es peor	Aceptar si u < p; u uniforme en [0,1)

Para Δ ≥ 0 se evalúa a = exp(−Δ/T) y p = a/(1+a), expresión equivalente que evita exponenciales positivas enormes cuando T es muy pequeña. La temperatura siempre permanece positiva. Al enfriarse, disminuye la posibilidad de aceptar empeoramientos.

Esquema de ejecución

```
vc = permutación_aleatoria(15)
actual = L(vc); mejor = vc.copy(); T = 100
mientras T >= 0.01:
    repetir 100 veces:
        vn = intercambio_aleatorio_de_dos_posiciones(vc)
        delta = L(vn) - actual
        p = 1 si delta < 0; en otro caso, logística(delta, T)
        si delta < 0 o uniforme(0,1) < p:
            vc = vn; actual = L(vn)
            si actual < L(mejor): mejor = vc.copy()
    T = 0.99 * T
devolver mejor
```

Se almacena la mejor ruta por separado de la actual: aceptar temporalmente una peor permite explorar y no borra el mejor resultado. Con este calendario se realizaron 917 niveles y 91 700 propuestas; después del último nivel T = 0.009941993 < 0.01.

Ejemplo realmente observado: el primer intercambio aumentó L de 631.089690 a 653.931080. Con T = 100, Δ = 22.841391, p = 0.443144 y u = 0.281838, se aceptó esa vecina peor porque u < p.

Resultado de la corrida principal

Con semilla 0, la longitud pasó de 631.089689696 a 288.609288133 unidades del plano. La reducción respecto a esa ruta inicial fue de 54.2681 %.

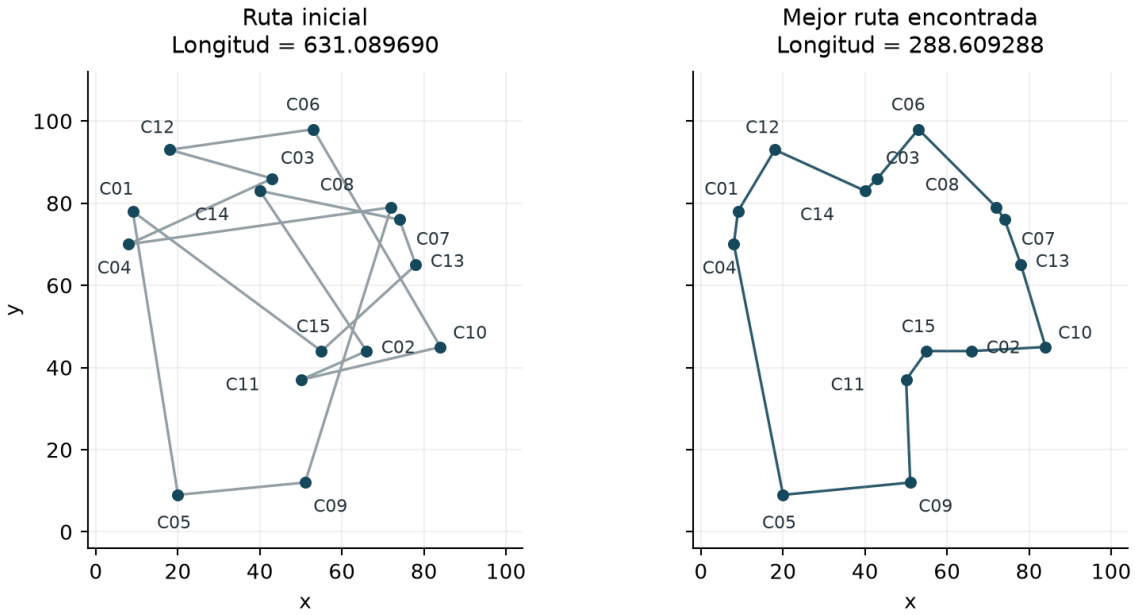


Figura 1. Los dos dibujos incluyen la arista que cierra el recorrido. Las coordenadas y la escala son las mismas; sólo cambia el orden de visita.

Medida	Valor observado
Longitud inicial	631.089689696
Mejor longitud	288.609288133
Reducción	54.2681 %
Niveles / vecinos propuestos	917 / 91 700
Aceptaciones totales	7596
Mejoras / empeoramientos aceptados	4265 / 3331
Empates aceptados	0
Óptimo de referencia Held–Karp	288.609288133

Mejor ciclo, presentado desde C01:

C01 → C04 → C05 → C09 → C11 → C15 → C02 → C10 → C13 → C07 → C08 → C06 → C03 → C14 → C12 → C01

Cambiar el punto de inicio de la escritura conserva el mismo ciclo y la misma longitud. El algoritmo permite intercambiar cualquier par de posiciones; no fija C01 durante la búsqueda.

Convergencia y verificación del resultado

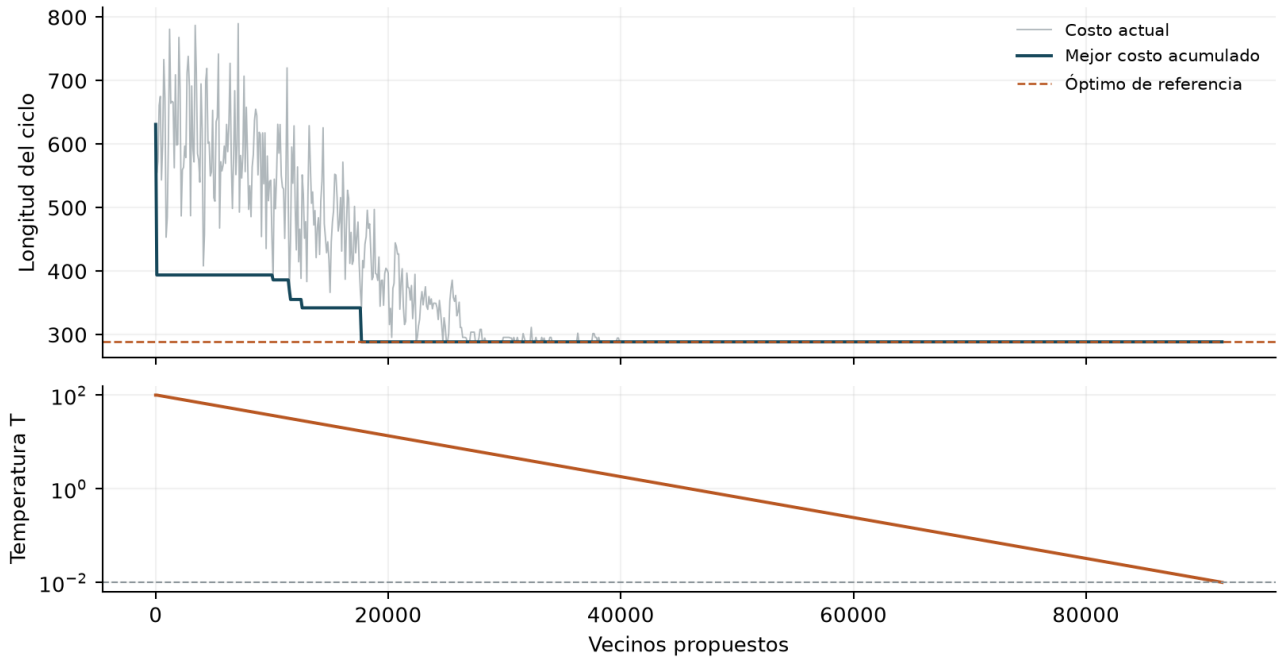


Figura 2. Valores registrados al terminar cada nivel. El costo actual puede subir por las aceptaciones probabilísticas; el mejor acumulado sólo disminuye o permanece constante. La gráfica inferior usa escala logarítmica para T.

La temperatura graficada es la usada en cada nivel. Tras el último se aplica otro enfriamiento y se alcanza el valor de parada. La corrida principal aceptó 3331 empeoramientos, evidencia de que la búsqueda no se limitó a aceptar mejoras.

Referencia exacta

Se resolvió la misma matriz por programación dinámica Held–Karp [3], obteniendo 288.609288133. El mejor costo del Templado Simulado coincide con esta referencia con tolerancia absoluta de 10^{-8}. El cálculo de referencia procesó 114688 subproblemas; además se contrastó su implementación con fuerza bruta sobre seis ciudades y las 120 permutaciones con origen fijo, coincidiendo ambos resultados.

En las 20 corridas con semillas 0–19, 20 alcanzaron ese costo óptimo de referencia para esta instancia. Es una frecuencia observada de este experimento; no garantiza el óptimo en otras corridas, datos o calendarios de enfriamiento.

El procedimiento satisface el enunciado: ciclo de 15 ciudades, vecindad por intercambio de dos posiciones y aceptación adaptada a minimización. La comparación exacta permite afirmar que el costo alcanzado es óptimo para esta matriz numérica, sin convertir al Templado Simulado en un método de garantía general.

Fuentes consultadas

[1] Material de clase, sin fecha. 8-BúsquedasNoInf-Inf y heuristicas modernas(TempSim).ppt. Láminas 27–29: aceptación y empates; 30–31: algoritmo y enfriamiento; 32: Tarea 5.

[2] MathWorks. How Simulated Annealing Works. Regla de aceptación; consulta: 3 de noviembre de 2026. https://www.mathworks.com/help/gads/how-simulated-annealing-works.html

[3] Held, M. y Karp, R. M. (1961). A dynamic programming approach to sequencing problems. ACM National Meeting. Ficha de IBM Research. https://research.ibm.com/publications/a-dynamic-programming-approach-to-sequencing-problems